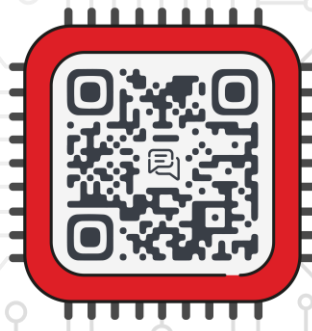




نظري

4

10

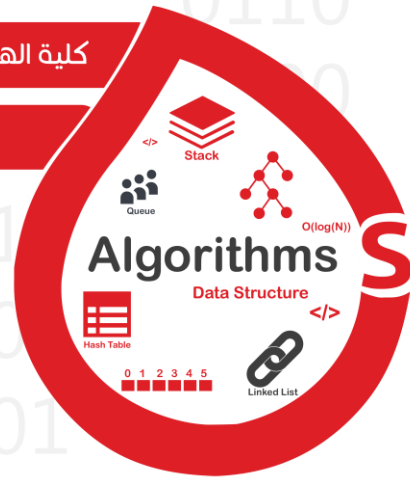
1200 sp

كلية الهندسة المعلوماتية

السنة الثانية

طرق التفكير 2

د. أبي صندوق



RB Informatics; 24/11/2024

خوارزميات 1

تحدثنا في المحاضرة السابقة عن أول طريقة من طرق التفكير وهي Decrease and conquer وستحدث اليوم عن ثاني طريقة وهي Divide and conquer .

ثانياً : Divide and conquer

وفيها نُجزئ المشكلة الأساسية لعدة أجزاء، ثم نحل كل مشكلة على حدى وعند الوصول لحل كل جزء نقوم بدمج هذه الحلول حتى نحصل على حل كامل للمسألة الأساسية .

مثال 1 :

(نظرياً)

لدينا كتاب تعرّض للتلف وتبعثرت أوراقه ، ونرغب في إعادة ترتيب الأوراق .

الحل :

- نقسم الأوراق إلى نصفين ونرتب كل نصف على حدى بطريقة ما
- نصبح بعدها أمام مشكلتين جزئيتين ، أي كل نصف لوحده مرتب ولكن نحتاج لدمجهم معاً .
- لدمجهم معاً في كل مرة نقارن الورقتين على سطح كل نصف ونختار الأصغر منهما ثم نكرر العملية حتى ننتهي من دمج القسمين معاً بشكل كامل . لتصبح جميع الأوراق مرتبة .

فمثلاً :

إذا كان لدينا 100 ورقة نستطيع تقسيمها إلى أربع مجموعات بطريقة ما..
و ثم نرتب كل قسم على حدى ومن ثم ندمجهم معاً حسب الطريقة السابقة وذلك بمقارنة الأرقام على أسطح الأقسام الأربعة المرتبة واختيار الأصغر منها ونضعه في المصفوفة وهكذا ، حتى نحصل على المجموعة الأساسية المرتبة .

ملاحظة :

في عملية الدمج في كل مرة قمنا بعدد عمليات ثابتة أي التعقيد خطي ($O(n)$)
حيث n : عدد الاوراق هنا (عدد العناصر المراد ترتيبها)

بفرض لدينا مصفوفة تحوي أرقام غير مرتبة ونريد ترتيبها تصاعدياً

A	6	2	3	4	1	9
---	---	---	---	---	---	---

الحل :

الخطوة الأولى : تجزئة المسألة

سنقسم المصفوفة لجزئين بطريقة ما ولتكن بالنصف مثلاً

A	6	2	3	4	1	9
---	---	---	---	---	---	---

نرتب كل قسم بطريقة ما (لا يشترط استعمال نفس طريقة الترتيب للقسمين) لنصل إلى هذه النتيجة

2	3	6	1	4	9
---	---	---	---	---	---

الخطوة الثانية : دمج الحلول

بما أن كلا القسمين مرتبين فالعنصر الأول في المصفوفة الكلية (العنصر الأصغر) إما العنصر الأصغر في القسم الأول أو العنصر الأصغر في القسم الثاني ، لذا نقارن بينهما :

- تتم المقارنة عن طريق وضع مؤشر في بداية كل قسم ونقوم بالمقارنة فإذا كان مثلاً : (المؤشر الأول < المؤشر الثاني) : نقوم بوضع رقم المؤشر الثاني في المصفوفة الأساسية (من البداية) ونزيح المؤشر الثاني (أي نجعله يشير إلى الرقم الذي بعده في المصفوفة الموجود فيها) ومن ثم نقارن رقم المؤشر الثاني الجديد مع رقم المؤشر الأول وهكذا بالمقارنة نجد $1 < 2$

2	3	6
---	---	---



1	4	9
---	---	---



1					
---	--	--	--	--	--

نكرر العملية وبمقارنة أصغر عنصرين نجد $2 < 4$

2	3	6
---	---	---



1	4	9
---	---	---



1	2				
---	---	--	--	--	--

نعيد تكرار العملية حتى نصل للترتيب الصحيح

1	2	3	4	6	9
---	---	---	---	---	---

تعقيد عملية الدمج merge في أسوأ الحالات :

- سنمر على كل عناصر المصفوفة لنقارنهم أي نحتاج n عملية
 - ولكل عنصر نحتاج كحد أقصى عملية مقارنة واحدة (قد نحتاج 0 عملية بحال انتهت أعداد أحد القسمين قبل الآخر ، لذا بقية العناصر بالقسم الآخر سيتم سحبهم بلا أي مقارنة)
 - ونحتاج n عملية إسناد لتشكيل المصفوفة المرتبة .
- وبالتالي تعقيد عملية الدمج $(2n)$ أي $O(n)$

نستنتج :

عملية الدمج merge خطية .

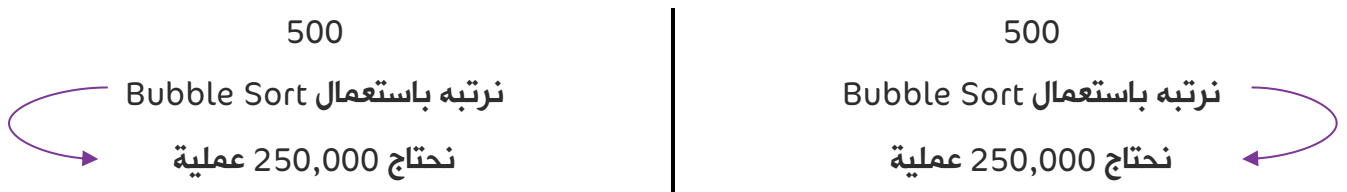
أي إذا كان لدينا سلسلتين مرتبتين ، نحتاج n عملية لدمجهم .

**هل يا ترى ترتيب المصفوفة بنفس الطريقة السابقة مفيد ؟**

لنفرض لدينا $n=1000$ عنصر

- إذا قمنا بترتيبهم باستعمال Bubble Sort سنحتاج تقريباً n^2 عملية أي $10^6 \approx$
- أو باستعمال insertion Sort سنحتاج تقريباً n^2 عملية أي $10^6 \approx$

- لنقسم العناصر لقسمين



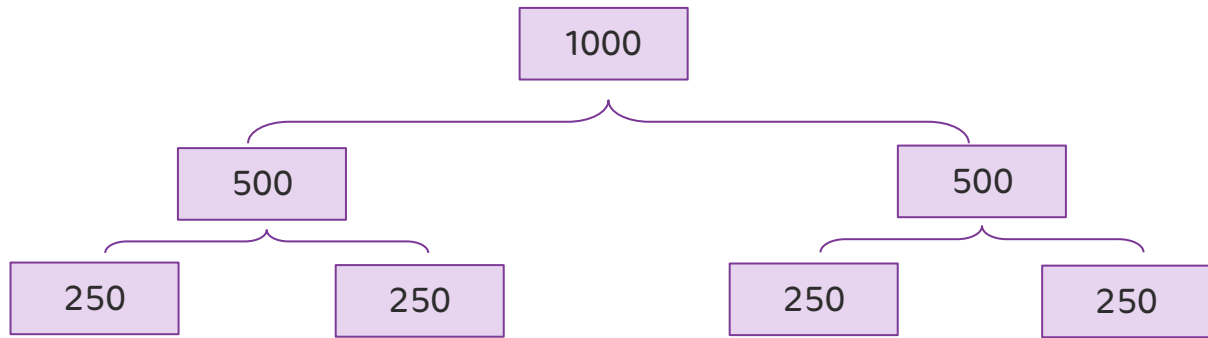
أي احتجنا 500,000 عملية لترتيب كل قسم لوحده دون عملية الدمج

- ولدمجهم نحتاج n عملية أي 1000

بالنتيجة احتجنا 501,000 عملية .

وبالتأكيد 501,000 عملية أفضل بكثير من 10^6 عملية

- ولو قمنا بتقسيم كل قسم بنفس الطريقة بدلاً من ترتيبهم باستعمال Bubble Sort مباشرة



ومن ثم رتبنا كل قسم باستعمال Bubble Sort سنحتاج إلى $4(250)^2 = 250,000$ عملية للترتيب و $n=1000$ عملية للدمج .

أي 251,000 عملية .

وهذا أفضل حتى من 501,000

الخوارزمية :

	Cost	Times
1. <code>void MergeSort (int []A,int st,int ed){</code>		
2. <code>if(ed-st < 1)</code>	C	1
3. <code>return ;</code>	C	0
4. <code>int m = (st+ed)/2;</code>	C	1
5. <code>MergeSort(A,st,m);</code>	$T(\frac{N}{2})$	1
6. <code>MergeSort(A,m+1,ed);</code>	$T(\frac{N}{2})$	1
7. <code>Merge(A,st,m+1,ed);</code>	$O(N)$	1
8. <code>}</code>		

🔗 شرح الكود :

السطر 1 : تابع MergeSort يستقبل مصفوفة من الأعداد الصحيحة int وعددين صحيحين من int هما st و ed يمثلان بداية ونهاية المصفوفة المراد ترتيبها .

السطر 2 و 3 : من أجل عنصر واحد فقط في المصفوفة تكون المصفوفة مرتبة (وهو شرط التوقف).

السطر 4 : متغير m يعبر عن نصف طول المصفوفة .

السطر 5 : نستدعي التابع نفسه (MergeSort) من أجل النصف الأول من المصفوفة .

السطر 6 : نستدعي التابع نفسه (MergeSort) من أجل النصف الثاني من المصفوفة .

■ ملاحظة :

في كلا السطرين الخامس والسادس يمكن استدعاء أي تابع ترتيب غير MergeSort ك Bubble Sort أو غيره ((مع الانتباه لوضع شرط التوقف المناسب)).

السطر 7 : نستدعي تابع الدمج Merge (بفرض كتبناه سابقاً) وهو تابع يقوم بدمج القسمين بعد ترتيبهما (في السطر الخامس والسادس) والحصول على مصفوفة مرتبة تحوي القسمين .

🔗 شرح التعقيد :

السطر 2 :

Times : سيتكرر مرة واحدة .

Cost : عملية مقارنة 1C

السطر 3 :

Times : بالحالة الأسوأ سيتنفذ 0 مرة [للتفصيل أكثر نعطي هذا السطر indicator أي أنه سيتنفذ x مرة وباقي الأسطر ستنفذ (1-x) مرة] ، أي إما سيتم تنفيذه أو تنفيذ باقي الأسطر .

Cost : 1C

السطر 4 :

Times : ينفذ مرة واحدة .

Cost : كلفته ثابتة 1C .

السطر 6+5 :

Times : ينفذ مرة واحدة.

Cost : لا نعلم كم يكلف هذا التابع ولكن نعلم أن الكلفة الزمنية هي نفس الكلفة الزمنية للتابع كامل $T(N)$ لكن بنصف عدد العناصر أي $T(\frac{n}{2})$.

انتبه!

$$T\left(\frac{n}{2}\right) \neq \frac{T(n)}{2}$$

الزمن اللازم لترتيب نصف عدد العناصر .

نصف الزمن اللازم لترتيب n عنصر

السطر 7:

Times : ينفذ مرة واحدة

Cost : يمكننا كتابة الكود وحساب تعقيده بدقة $T(\dots)$ ولكن مهما كان التعقيد فنحن نعلم مسبقاً أن تعقيد الـ merge من رتبة n ، أي يمكن كتابة $O(n)$ مباشرة .

التعقيد الكلي = عدد العمليات $\times$ الزمن اللازم لتنفيذ كل عملية .

$$T(N) = C + (0)C + C + 2 T\left(\frac{N}{2}\right) + O(N)$$

ولكن $C+C=2C$ وهي ثوابت ، ولدينا $O(N)$ و الثابت يهمل ضمن الـ $O(N)$

$$T(N) = 2 T\left(\frac{N}{2}\right) + O(N) \quad \text{sec}$$

تابع رياضي عودي

■ ملاحظة :

بأفضل الحالات $O(1)$ لأننا سنحتاج عمليتين فقط للسطر الثاني والثالث (وذلك بفرض وجود عنصر واحد فقط).



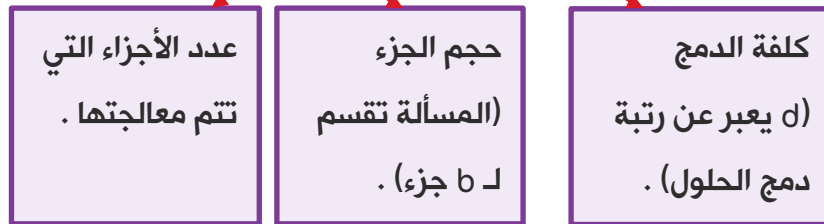
Nothing is impossible the word
It self says "I'm possible"

التحويل من تابع رياضي عودي إلى غير عودي (Normal form)

النظرية الأساسية Master Theorem

✍ إذا كان لدينا تابع زمني من الشكل :

$$T(N) = a T\left(\frac{N}{b}\right) + O(N^d) \text{ sec}$$



نوجد a و b^d ثم نقارن بينهم :

1. $a < b^d$ عدد قليل من الأجزاء وكلفة الدمج عالية .

فالتعقيد الأساسي يأتي من الدمج النهائي (آخر عملية دمج)

$$T(N) \in O(N^d)$$

2. $a > b^d$ عدد كبير من الأجزاء و زمن الدمج مهمل .

فالتعقيد الأساسي يأتي من عملية التجزئ .

$$T(N) \in O(N^{\log_b(a)})$$

3. $a = b^d$ حالة من التساوي بين عدد الأجزاء وكلفة الدمج .

$$T(N) \in O(N^d * \log_b(N))$$

✍ بالعودة إلى حالتنا : $a=2, b=2, d=1$

$$a=2 = b^d = 2^1=2$$

$$2 = 2$$

بالتعويض :

$$T(N) \in O(N * \log_2(N))$$

وبذلك يكون تعقيد الـ MergeSort هو : $O(n \log(n))$

مثال 2:

✍ تحدثنا عن الـ Quick select في المحاضرة السابقة .

عندما حاولنا إيجاد العنصر صاحب الترتيب K وذلك بتجريب عناصر عشوائية ولكل عنصر نضع العناصر الأصغر منه قبله

والأكبر منه بعده في مكانة الصحيح ونكرر العملية حتى نجد العنصر صاحب الترتيب k .

🔗 سنتحدث الآن عن Quick Sort :

نفرض لدينا المصفوفة الآتية :

A	7	3	15	5	10	12	8	9	13	1	4
---	---	---	----	---	----	----	---	---	----	---	---

1. نأخذ عنصر عشوائي وليكن الأول مثلاً ، ونضع مؤشر في البداية ومؤشر في النهاية .

A	7	3	15	5	10	12	8	9	13	1	4
	Pivot	i								j	

2. نستمر بتحريك المؤشر (i) حتى نجد عنصر أكبر من 7 فنجد $7 < 15$ ثم نستمر بتحريك المؤشر (j) حتى نجد عنصر أصغر من 7 فنجد $7 > 4$

A	7	3	15	5	10	12	8	9	13	1	4
		i								j	

نبدل بينهما

A	7	3	4	5	10	12	8	9	13	1	15
		i								j	

3. نكرر العملية فنجد

▪ عند تحريك (i) $7 < 10$ ▪ وعند تحريك (j) $7 > 1$

A	7	3	4	5	10	12	8	9	13	1	15
					i					j	

نبدل بينهما

A	7	3	4	5	1	12	8	9	13	10	15
					i					j	

4. نكرر العملية فنجد:

▪ عند تحريك (i) $7 < 12$

▪ وعند تحريك (j) أن (j) قد تجاوز (i) وهذا يعني أن المكان الواقف j عنده

كل ما قبله أصغر من 7 ، وكل ما بعده أكبر من 7

A	7	3	4	5	1	12	8	9	13	10	15
					j	i					

لذا نبدل الآن بين الـ (7) و $A[j]$

A	1	3	4	5	7	12	8	9	13	10	15
---	---	---	---	---	---	----	---	---	----	----	----

نرتبها

نرتبها

هكذا نكون أوجدنا المكان الصحيح للعنصر 7 و لإكمال ترتيب المصفوفة نعيد الخطوات على كل جزء أو نرتب كل جزء على حدى

بفرض رتبنا كل قسم على حدى بطريقة bubble sort ، فلن نحتاج عملية دمج لأن المصفوفة ستكون مرتبة أصلاً .. وبالتالي نعتبر تعقيد الدمج $O(1)$.
وفي حال أردنا أن ندمج (أي نعيد نسخ العناصر) فتعقيد الدمج $O(n)$

لحساب التعقيد بأسوأ الحالات :

1. عملية قسم المصفوفة لقسمين (Quick Select) هي $O(n)$
2. ترتيب كل قسم لوحده باستعمال bubble sort :
- بالحالة الأسوأ n^2 (ال Pivot كان ترتيبه الصحيح بدايتها أو نهايتها).
- بالحالة الأفضل $\left(\frac{n}{2}\right)^2$ انقسمت المصفوفة لقسمين متساويين.
3. عملية الدمج $O(n)$
4. التعقيد بين $\left(\frac{n}{2}\right)^2$ إلى n^2

في حال رتبنا كل قسم باستعمال التابع نفسه يكون التعقيد $n \log(n) \leftarrow n^2$

بالعودة لمثال الكتاب ذو الأوراق المبعثرة (بفرض الكتاب 100 صفحة)

الحل:

نجمع الأوراق تحت الـ 50 معاً فوق الـ 50 معاً ، ثم نرتب كل قسم منهما وندمجهم

- عملية التقسيم n عملية
- الدمج بأسوأ الحالات n عملية
- الترتيب بـ Bubble Sort هو $\left(\frac{n}{2}\right)^2$
- بـ Merge Sort هو $n \log(n)$
- بفرض رتبنا بـ Merge Sort : $n \log(n) = n + n + n \log(n)$

- لتقليل التعقيد يمكن إعادة نفس الخطوات على كل قسم بدلاً من ترتيبه باستعمال Bubble Sort
- بالتجريب أو الإحصاء فإن احتمال ظهور الحالة الوسطى (مثلاً قسم المصفوفة إلى $\frac{1}{4}$ و $\frac{3}{4}$) جيد لكن الحالة الحدية (ظهور العنصر بالوسط تماماً أو على الأطراف) احتمالها قليل .
(كلما اقتربنا من الأطراف كلما قلّ الاحتمال).
- أغلب توابع الـ Sort بلغات البرمجة تطبق الـ Quick Sort كما أن إدارتها للذاكرة أفضل من أي خوارزمية ثانية من نفس الرتبة .
(تعقيدها بأفضل الحالات هو $O(n \log(n))$).

خلاصة

- لو كان لدينا أجزاء من المسألة محلولة أو نستطيع تقسيمها لأجزاء وحل كل جزء
- إذا كنا نستطيع دمج الحلول لنوجد الحل النهائي إذاً يوجد حل بطريقة divide and conquer قد تكون فعالة الطريقة وقد لا تكون.
- وإذا قسمنا الأجزاء بطريقة ممنهجة ذكية سنوجد حل efficient فعال .

	Best Case	Worst Case
Quick Select	$O(n)$	$O(n^2)$
Merge	$O(n)$	$O(n)$
Merge Sort (باستدعاء تابع عودي للتابع نفسه)		$O(n \log_b n)$
Quick Sort (إذا استعملنا Merge Sort لترتيب الأقسام)		$O(n \log_b n)$
Quick Sort (إذا استعملنا Bubble Sort لترتيب الأقسام)	$O(\frac{n}{2})^2$ إذا كان الـ pivot يقسم المصفوفة لقسمين بالتساوي	$O(n^2)$ إذا كان الـ pivot أكبر أو أصغر عنصر
Quick Sort (باستدعاء تابع عودي للتابع نفسه)	$O(n \log(n))$ في حال كان الـ pivot يقسم المصفوفة لقسمين بالتساوي	$O(n^2)$ عندما يتم اختيار أكبر أو أصغر عنصر كـ Pivot دوماً

The end